

Производные типы в Go

Именованные типы и производные

Именованные типы в Go позволяют создать новый тип на основе существующего типа. Это полезно для повышения читаемости кода и создания более специфичных типов для определенных целей.

```
package main

import "fmt"

// Age
// Создаем именованный тип на основе int
type Age int

func main() {
    var myAge Age = 25
    fmt.Println(myAge)
}
```

Функция как тип данных

В Go функции являются первоклассными гражданами, что означает, что их можно присваивать переменным, передавать в качестве аргументов и возвращать из других функций.

```
package main

import "fmt"

// Adder
// Определяем тип функции
type Adder func(int, int) int

// Реализуем функцию, соответствующую типу Adder
func add(a int, b int) int {
    return a + b
}

func main() {
    var myAdder Adder = add
    result := myAdder(2, 3)
    fmt.Println(result)
}
```

Псевдонимы

Псевдонимы типов позволяют создать альтернативное имя для существующего типа. Они не создают новый тип, а просто дают другое имя для существующего типа.

```
package main

import "fmt"

// MyInt
// Создаем псевдоним для типа int
type MyInt = int

func main() {
    var x MyInt = 10
    fmt.Println(x)
}
```

Структуры

Структуры в Go используются для объединения данных разных типов в одну сущность. Поля структуры могут быть разного типа, что позволяет хранить связанные данные вместе.

```
package main

import "fmt"
```

```
// Person
// Определяем структуру
type Person struct {
    Name string
    Age  int
}

func main() {
    // Создаем и инициализируем структуру
    p := Person{Name: "Alice", Age: 30}
    fmt.Println(p)
}
```

Создание и инициализация структур

Структуры можно создавать и инициализировать несколькими способами.

- Использование литерала структуры:

```
p := Person{Name: "Alice", Age: 30}
```

- Без указания имен полей (порядок важен):

```
p := Person{"Alice", 30}
```

- Создание структуры с использованием ключевого слова new:

```
p := new(Person)
p.Name = "Alice"
p.Age = 30
```

- Создание структуры через указатель:

```
p := &Person{Name: "Alice", Age: 30}
```

Обращение к полям структуры

Обращение к полям структуры осуществляется с помощью оператора точки (.).

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func main() {
    p := Person{Name: "Alice", Age: 30}
    fmt.Println(p.Name)
    fmt.Println(p.Age)
}
```

Указатели на структуры

Указатели на структуры позволяют изменять значения полей структуры без необходимости создания копии структуры.

```
package main

import "fmt"

type Person struct {
    Name string
    Age  int
}

func main() {
    p := &Person{Name: "Alice", Age: 30}
    p.Age = 31
    fmt.Println(p)
}
```

Вложенные структуры

Структуры могут содержать другие структуры в качестве полей, что позволяет создавать более сложные данные.

```
package main

import "fmt"

type Address struct {
    City   string
    State  string
}

type Person struct {
    Name    string
    Age     int
    Address Address
}

func main() {
    p := Person{
        Name: "Alice",
        Age:  30,
        Address: Address{
            City: "New York",
            State: "NY",
        },
    }
    fmt.Println(p)
}
```

Хранение ссылки на структуру того же типа

Структуры могут содержать указатели на другие структуры того же типа, что позволяет создавать связные структуры данных, такие как деревья или графы.

```
package main

import "fmt"

type Node struct {
    Value int
    Next  *Node
}

func main() {
    n1 := &Node{Value: 1}
    n2 := &Node{Value: 2}
    n3 := &Node{Value: 3}

    n1.Next = n2
    n2.Next = n3

    // Печатаем значения узлов
    current := n1
    for current != nil {
        fmt.Println(current.Value)
        current = current.Next
    }
}
```

В данном примере мы создаем простую односвязную структуру, где каждый узел (Node) содержит значение и указатель на следующий узел. Это позволяет нам хранить ссылки на другие структуры того же типа, создавая цепочку узлов.

Анонимные структуры

Анонимные структуры позволяют создавать структуры без явного определения типа. Они полезны для создания временных структур или структур, используемых в ограниченном контексте.

```
package main

import "fmt"

func main() {
    person := struct {
```

```
    Name string
    Age  int
} {
    Name: "Alice",
    Age:  30,
}
fmt.Println(person)
}
```